

Task Flow Saas

Versión 1.0

Autor: Diego Francisco Hernández Ramírez

Fullstack Project Documentation

Fecha: 23 de marzo del 2026



Introducción

TaskFlow SaaS es una aplicación web de gestión de proyectos y tareas inspirada en herramientas como Trello y Jira. El objetivo principal fue desarrollar una aplicación Full Stack moderna utilizando tecnologías actuales de frontend, backend y bases de datos, además de desplegarla completamente en producción.

Este proyecto fue desarrollado de forma autodidacta, investigando documentación oficial, resolviendo errores reales de desarrollo y aprendiendo a integrar múltiples tecnologías dentro de una misma solución.

Más allá de implementar funcionalidades, el proyecto representó una oportunidad para enfrentar problemas reales relacionados con arquitectura web, despliegue, autenticación, configuración de servidores y bases de datos.

Introduction

TaskFlow SaaS is a project and task management web application inspired by tools like Trello and Jira. The main objective was to develop a modern Full Stack application using current frontend, backend and database technologies, in addition to deploying it completely in production.

This project was developed self-taught, researching official documentation, solving real development errors and learning to integrate multiple technologies within the same solution.

Beyond implementing functionalities, the project represented an opportunity to face real problems related to web architecture, deployment, authentication, server and database configuration.

Objetivos del Proyecto

Los objetivos principales fueron:

- Construir una aplicación Full Stack moderna.
- Implementar autenticación segura mediante JWT.
- Diseñar una API REST funcional.
- Utilizar una base de datos relacional en producción.
- Implementar operaciones CRUD completas.
- Desplegar frontend y backend en servicios cloud.
- Resolver problemas reales de integración y despliegue.
- Simular un flujo de trabajo similar al utilizado en proyectos profesionales.

Project Objectives

The main objectives were:

- Build a modern Full Stack application.
- Implement secure authentication using JWT.
- Design a functional REST API.
- Use a relational database in production.
- Implement complete CRUD operations.
- Deploy frontend and backend in cloud services.
- Solve real integration and deployment problems.
- Simulate a workflow similar to that used in professional projects.

Tecnologías Utilizadas / Technologies Used

FRONTEND

- React
- TypeScript
- Vite
- Tailwind CSS
- Axios
- DnD Kit

BACKEND

- Node.js
- Express
- TypeScript
- Prisma ORM
- JWT

BASE DE DATOS / DATABASE

- PostgreSQL
- Neon

DESPLIEGUE / DEPLOYMENT

- Render

CONTROL DE VERSIONES / VERSION CONTROL

- Git
- GitHub

Arquitectura General

Usuario



Frontend React



Axios



API REST Express



Prisma ORM



PostgreSQL (Neon)

El frontend consume una API REST desarrollada en Express.

La API se comunica con PostgreSQL utilizando Prisma ORM.

La autenticación se realiza mediante JSON Web Tokens (JWT).

Tanto frontend como backend se encuentran desplegados en producción utilizando Render.

General Architecture

User



Frontend React



Axios



REST Express API



Orm prism



PostgreSQL (Neon)

The frontend consumes a REST API developed in Express.

The API communicates with PostgreSQL using Prisma ORM.

Authentication is done using JSON Web Tokens (JWT).

Both frontend and backend are deployed in production using Render.

Funcionalidades Implementadas

GESTIÓN DE USUARIOS

- Registro de usuarios
- Inicio de sesión
- Autenticación JWT
- Persistencia de sesión

GESTIÓN DE PROYECTOS

CRUD COMPLETO

- Crear proyectos
- Consultar proyectos
- Editar proyectos
- Eliminar proyectos

GESTIÓN DE TAREAS

CRUD COMPLETO

- Crear tareas
- Consultar tareas
- Editar tareas
- Eliminar tareas

ORGANIZACIÓN KANBAN

Estados disponibles:

- TODO
- IN_PROGRESS
- DONE

DRAG & DROP

Las tareas pueden moverse entre columnas mediante arrastrar y soltar.

Esta funcionalidad fue implementada utilizando DnD Kit.

INTERFAZ

- Diseño responsive
- Tema claro
- Tema oscuro

Implemented Features

USER MANAGEMENT

- User registration
- Login
- Jwt authentication
- Session persistence

PROJECT MANAGEMENT

COMPLETE CRUD

- Create projects
- Consult projects
- Edit projects
- Delete projects

TASK MANAGEMENT

COMPLETE CRUD

- Create tasks
- Consult tasks
- Edit tasks
- Delete tasks

KANBAN ORGANIZATION

AVAILABLE STATES:

- ALL
- IN_PROGRESS
- DONE

DRAG & DROP

Tasks can be moved between columns using drag and drop.

This functionality was implemented using DnD Kit.

INTERFACE

- Responsive design
- Clear topic
- Dark theme

Despliegue en Producción

BACKEND

Desplegado en Render.

Incluye:

- Variables de entorno
- Prisma
- Conexión a PostgreSQL
- Configuración de CORS

BASE DE DATOS

Se utilizó PostgreSQL hospedado en Neon.

Características:

- Base de datos en la nube
- Conexión mediante Prisma
- Persistencia de datos en producción

FRONTEND

Desplegado en Render.

Configurado para:

- Consumir la API en producción
- Manejo de variables de entorno
- Compilación automática mediante GitHub

Deployment in Production

BACKEND

Deployed in Render.

Includes:

- Environment variables
- Prism
- Connection to PostgreSQL
- CORS configuration

DATABASE

PostgreSQL hosted on Neon was used.

Features:

- Cloud database
- Connection via Prism
- Persistence of data in production

FRONTEND

Deployed in Render.

Configured for:

- Consume the API in production
- Management of environment variables
- Automatic compilation using GitHub

Variables de Entorno / Environment Variables

BACKEND

DATABASE_URL=

JWT_SECRET=

PORT=

FRONTEND

VITE_API_URL=

Retos Técnicos Encontrados

Durante el desarrollo se presentaron diversos problemas técnicos similares a los que aparecen en proyectos profesionales.

PRISMA Y POSTGRESQL

Se presentaron problemas relacionados con:

- Configuración de Prisma
- Migraciones
- Tablas inexistentes
- Sincronización del esquema

Aprendizajes:

- Generación de migraciones
- Prisma Client
- Gestión de esquemas de base de datos

Technical Challenges Found

During development, various technical problems similar to those that appear in professional projects arose.

PRISM AND POSTGRESQL

Problems related to:

- Prism Configuration
- Migrations
- Non-existent tables
- Schema synchronization

Learning:

- Generation of migrations
- Prism Client
- Database schema management

TypeScript

Se encontraron errores relacionados con:

- Tipado de parámetros
- Tipado de rutas Express
- Tipado de respuestas API
- Compatibilidad entre frontend y backend

Aprendizajes:

- Interfaces
- Tipos personalizados
- Validación estática

TypeScript

Errors related to:

- Parameter typing
- Express route typing
- API response typing
- Frontend and backend compatibility

Learning:

- Interfaces
- Custom types
- Static validation

CORS

Durante el despliegue el frontend no podía comunicarse con el backend.

Problema:

Blocked by CORS policy

Solución:

- Configuración de dominios permitidos
- Configuración correcta de Express y Render

CORS

During deployment the frontend could not communicate with the backend.

Problem:

Blocked by CORS policy

Solution:

- Configuration of allowed domains
- Correct Express and Render configuration

Render

Se encontraron problemas relacionados con:

- Variables de entorno
- Comandos de build
- Configuración de servicios
- Despliegues fallidos

Aprendizajes:

- Logs de producción
- Diagnóstico de errores
- Configuración de servicios cloud

Render

Problems were found related to:

- Environment variables
- Build commands
- Service configuration
- Failed deployments

Learning:

- Production logs
- Error diagnosis
- Cloud services configuration

React Router

Se presentaron problemas relacionados con:

- Rutas protegidas
- Navegación
- Deploy en producción

Aprendizajes:

- Protección de rutas
- Manejo de autenticación

React Router

Problems related to:

- Protected routes
- Navigation
- Deploy in production

Learning:

- Route protection
- Authentication management

Tailwind CSS

Se encontraron problemas relacionados con:

- Configuración inicial
- Compatibilidad con Vite
- Configuración de PostCSS

Aprendizajes:

- Integración moderna de Tailwind
- Optimización de estilos

Tailwind CSS

Problems were found related to:

- Initial configuration
- Vite compatibility
- PostCSS Configuration

Learning:

- Modern Tailwind integration
- Style optimization

Aprendizajes Obtenidos

Este proyecto fue desarrollado de manera autodidacta.

Además de aprender nuevas tecnologías, el principal aprendizaje fue desarrollar la capacidad de investigar, diagnosticar y resolver problemas técnicos de forma independiente.

Durante el proyecto fue necesario:

- Leer documentación oficial.
- Analizar mensajes de error.
- Comprender logs de producción.
- Corregir problemas de despliegue.
- Integrar múltiples tecnologías.
- Adaptarse a cambios y errores inesperados.

Estas experiencias permitieron adquirir habilidades que son fundamentales en entornos profesionales de desarrollo de software.

Learning Obtained

This project was developed self-taught.

In addition to learning new technologies, the main learning was developing the ability to independently investigate, diagnose and solve technical problems.

During the project it was necessary:

- Read official documentation.
- Scan error messages.
- Understand production logs.
- Fix deployment issues.
- Integrate multiple technologies.
- Adapt to unexpected changes and errors.

These experiences allowed us to acquire skills that are essential in professional software development environments.

Resultados

Se logró construir y desplegar exitosamente una aplicación SaaS funcional que incluye:

- ✓ Backend con Express + TypeScript
- ✓ Base de datos PostgreSQL en Neon
- ✓ Prisma ORM
- ✓ Autenticación con JWT
- ✓ Frontend con React + TypeScript
- ✓ Drag & Drop de tareas
- ✓ CRUD de proyectos
- ✓ CRUD de tareas
- ✓ Edición de tareas
- ✓ Deploy del backend en Render
- ✓ Deploy del frontend en Render
- ✓ Conexión frontend ↔ backend en producción
- ✓ Configuración de CORS
- ✓ Variables de entorno
- ✓ Resolución de errores de compilación y despliegue

Results

A functional SaaS application was successfully built and deployed that includes:

- ✓ Backend with Express + TypeScript
- ✓ PostgreSQL database in Neon
- ✓ ORM Prism
- ✓ Authentication with JWT
- ✓ Frontend with React + TypeScript
- ✓ Drag & Drop tasks
- ✓ Project CRUD
- ✓ Task CRUD
- ✓ Editing tasks
- ✓ Deploy of the backend in Render
- ✓ Deploy of the frontend in Render
- ✓ Frontend connection ↔ backend in production
- ✓ CORS configuration
- ✓ Environment variables
- ✓ Resolving compilation and deployment errors

Conclusión

TaskFlow SaaS representa mi segundo proyecto Full Stack desplegado completamente en producción utilizando tecnologías modernas del ecosistema JavaScript y TypeScript.

El proyecto no solo me permitió aprender herramientas y frameworks ampliamente utilizados en la industria, sino también desarrollar habilidades de resolución de problemas, depuración y despliegue que forman parte del trabajo diario de un desarrollador profesional.

Este proyecto demuestra mi capacidad para aprender de forma autodidacta, enfrentar desafíos técnicos reales y construir soluciones funcionales de principio a fin.

Conclusion

TaskFlow SaaS represents my second Full Stack project deployed fully in production using modern technologies from the JavaScript and TypeScript ecosystem.

The project not only allowed me to learn tools and frameworks widely used in the industry, but also develop problem-solving, debugging and deployment skills that are part of the daily work of a professional developer.

This project demonstrates my ability to learn self-taught, face real technical challenges, and build functional solutions from start to finish.